Understanding and Uncovering the Spread of Covid-19 Today

April 29, 2022

Ananya Shah • shah.anan@northeastern.edu

DS 2001 - Spring 2022

Data Source: There are several data sets used in this project. They are as follows: 1. Comprehensive Covid Data: Website and was accessed on 28th April from this link:https://github.com/owid/covid-19-data/tree/master/public/data. 2. Variant Data: Website and was accessed on 28th April from this link:https://www.gisaid.org/epiflu-applications/influenza-genomic-epidemiology/ 3. Country and City Names: Website is an API database which can be found here:https://documenter.getpostman.com/view/1134062/T1LJjU52#dd5bd0d9-2602-4161-8c77-3af30cd2f41a 4. Covid 19 International Travel Restriction Policy Data: and was accessed on 28th April from this link:https://github.com/owid/covid-19-data/tree/master/public/data.

1 Abstract

The aim of the project was to idenify the new epicenters of COVID-19 using population, infection rates, and government response data amongst a few other things. To build a deeper understanding of the epicenters, after identifying them, the project also aims to predict in which of those epicenters a new variant of the disease may begin. This was predicted using historical data of cases, and present data of vaccinations, healthcare systems and current cases. And lastly, the main aim was awareness. As we begin to settle back into our normal lives, travel has also picked up quite fast. The project assigns a safety rating to each country based on their healthcare, economy, policies and current cases. This safety rating our of 100 tells you how safe a country is to travel to, 100 being the safest.

2 Background

The pandemic has exhausted us all. It has disrupted billions of lives. As the economy tries to move past the health disruptions caused by the pandemic, people are being desensitised to the disease. They aren't taking as many precautions as they would, especially with the mask mandate being lifted in major countries like the US.

However, the pandemic is far from over. The numbers of deaths may not be that high, but they are still present! Each day, someone in the world is passing away due to covid. The daily cases have also spiked in many countries. As we resume our lives, activities like travel and gatherings resume. These two are the most dangerous ways of spreading the virus. And now, with vaccines, herd immunity, and life back to normal, the threat of Covid-19 variants just got higher.

The aim of the project is to spread awareness about the existing waves of Covid, and how to live within its bounds. Despite working with mnay datasets, my research was limited due to the large scope of the subject. So, I have focused my efforts on promoting safe travel - giving a safety rating to each nation, and recommending only the safest countries to travel to. In my final recommendation, I highlight the safest countries. Their safety rating signifies that if you choose to travel to these nations your risk of catching covid, especially if you're vacinated, is rare to none.

3 Reading and Cleaning Data

[1]:
```
# Importing all packages
import csv
import requests
from country import Country
import json
from location import location
import pandas as pd
import matplotlib.pyplot as plt
import plotly.graph_objects as go
import plotly.express as px
import plotly.io as pio
from plotly.subplots import make_subplots
from plotly.offline import plot, iplot
from plotly.offline import init_notebook_mode as pyo
import requests
import json
import random
```

[2]:
```
def read_csv(filename):
    '''
    Function: read_csv
    Param: filename - which file should it read?
    Returns : a list of lists
    '''
    data =[]
    with open(filename, 'r') as infile:

        csvfile = csv.reader(infile, delimiter = ",")
        next(csvfile)
        for row in csvfile:
            data.append(row)


        return data
```

[3]:
```
def clean_string(input_st):
    ''' Function: clean_string
```

```
        Parameter: strings
        Returns: new version of the string, cleaned up
    '''
    s = ""
    string = ""
    for st in input_st:

        for char in st:

            if char.isalpha():
                s += char.lower()

        # Creating new list with cleaned up strings
            string += s
        s = ""
```

[4]:
```
total = "covid.csv"
covid = read_csv(total)
```

Use of Objects: I have created a class Country (which can be found in the file) which has Covid-19 attributes of the country that I want to use and evaluate. The function below converts my raw 2D list data to systematic and dyanamic objects that I can use holistcally.

[5]:
```
def obj(lol, loc):
    '''
    creates a list of country objects with just names

    Parameters
    ----------
    lol : list
        list of lists - each list is a row of data .

    Returns
    -------
    list of objects.

    '''
    # temp dictionary
    country = {}
    countries = []

    exception = ["World"]
    for data in lol:


        if data[1] != "":
```

```
            code = data[2]


        if code not in country.keys():
            country[code] = {}

        if data[5] != "":

            country[code]["New Cases"] = data[5]

        if data[6] != "":
            country[code]["New cases Weekly Smoothed"] = data[6]

        if data[7] != "" and data[4] != "":
            country[code]["Fatality Rate"] = float(data[7]) /  float(data[4])
            country[code]["Total Cases"] = data[4]

        if  data[40] != "":
            country[code]["Total Vaccinations Per 100"] = data[40]

        if  data[42] != "":
            country[code]["People Fully Vaccinated Per 100"] = data[42]

        if  data[43] != "":
            country[code]["Total Boosters Per 100"] = data[43]

        if  data[47] != "":
            country[code]["Govt"] = data[47]

        if  data[60] != "":
            country[code]["Beds"] = data[60]

        if  data[62] != "":
            country[code]["HDI"] = data[62]


        for row in loc:
            if code == row["long_name"]:
                country[code]["Location"] = [row["center_lat"],␣
↪row["center_lng"]]


    for k, v in country.items():



        c = Country(k)
```

```
        for i, j in v.items():
            if i == "New Cases":
                c.new_cases = j
            elif i == "New cases Weekly Smoothed":
                c.new_cases_weekly = j
            elif i == "Fatality Rate":
                c.fatality_r = j
            elif i == "Total Vaccinations Per 100":
                c.total_vax = j
            elif i == "People Fully Vaccinated Per 100":
                 c.fully_vax = j
            elif i == "Total Boosters Per 100":
                 c.booster = j
            elif i == "Govt":
                c.govt = j
            elif i == "Beds":
                c.beds = j
            elif i == "HDI":
                c.hdi = j
            elif i == "Location":
                c.lats = j
            elif i == "Total Cases":
                c.total_cases = j



        # Creating list of objects
        countries.append(c)


    return countries
```

This is a bit of sticky coding, after manually identifying which regions there happens to be an overlap, I designed a method to combine both their attributes. This is a special case for US and US Virgin Islands.

[6]:
```
def update(country_1, country_2, countries):
    '''
    Takes in a list of objects, and calculates the
    countries with the highest new cases

    Parameters
    ----------
    countries : list of objects
        List of Country Objects.

    country_1, country_2: Strings
```

```
    name of countries

    Returns
    -------
    List of Country Objects which could be considered epicenters.

    '''



    epicenters = []
    cases = []
    s = 0


    # Adjusting for USA & USA Islands
    c1 = Country(country_1)
    c2 = Country(country_2)
    c1_ind = countries.index(c1)
    c2_ind = countries.index(c2)

    combined_cases = float(countries[c1_ind].new_cases) +␣
↪float(countries[c2_ind].new_cases)
    combined_cases_w = float(countries[c1_ind].new_cases_weekly) +␣
↪float(countries[c2_ind].new_cases_weekly)
    c_total = float(countries[c1_ind].total_cases) + float(countries[c2_ind].
↪total_cases)
    ft = (countries[c1_ind].fatality_r + countries[c2_ind].fatality_r) / 2
    tv = (float(countries[c1_ind].total_vax) + float(countries[c2_ind].
↪total_vax)) / 2
    fv = (float(countries[c1_ind].fully_vax) + float(countries[c2_ind].
↪fully_vax)) / 2
    b = (float(countries[c1_ind].booster) + float(countries[c2_ind].booster)) /␣
↪2
    g = (float(countries[c1_ind].govt) + float(countries[c2_ind].govt)) / 2
    bed = (float(countries[c1_ind].beds) + float(countries[c2_ind].beds)) / 2
    hd = (float(countries[c1_ind].hdi) + float(countries[c2_ind].hdi)) / 2
    loc = countries[c1_ind].lats


    countries[c1_ind] = Country(country_1, combined_cases, combined_cases_w,␣
↪c_total,
                                tv, fv, b, ft, g, bed , hd, loc)

    countries.pop(c2_ind)
```

```
    return countries
```

Location is a lengthy dictionary with the geo location of each country. I was using this in a earlier iteration of the project when I was employing matplotlib for my graphs. Plotly had an inbuilt dataset for graphing on the world map, so I used that after switching.

[7]:
```
loc = location
data = obj(covid, loc)
updated = update("United States", "United States Virgin Islands", data)
```

4 Computing first objective: Calculating Major epicenters of Covid-19 today

This functions qualifies if a country object is an epicenter or not.

[8]:
```
def epicenter(country):
    '''
    Takes in a object and returns a boolean - if its a epicenter or not

    Parameters
    ----------
    country : Object
        Country object

    Returns
    -------
    boolean

    '''

    epicenter = False

    if (float(country.new_cases) > 20000 and
        float(country.total_cases) > 2000000 and
        (country.fatality_r >= 0.01)
        and float(country.new_cases_weekly) > 16000):
        epicenter = True
        print(country)

    return epicenter
```

This functions computes other required variables for the graphing epicenters. It is a series of detailed if-elif statements whose parameters were found using the function describe() on data frame created on the list of objects.

[9]:
```
def draw_world(world):
    '''
    takes in a list of objects

    Parameters
    ----------

    world : list of objects
        list of objects.

    Returns
    -------
    returns epicenter status and attributes for plottijng on map

    '''


    epicenters = []


    for country in world:



        if epicenter(country) == True:
            epicenters.append(country)

        # Setting marker & color size according to cases
        # These landmaerk number are gotten using decribe function
        # which gives the median breakdown
        if (float(country.new_cases) > 27000 and float(country.
↪new_cases_weekly) > 16000):
            marker = 1000
            country.size = marker
            color_sel = "darkred"
            a = 0.6
            label = country.name



        elif (float(country.new_cases) > 25000 and float(country.new_cases) <␣
↪27000):
             marker = 900
             country.size = marker
             color_sel = "crimson"
             country.color = color_sel
             a = 0.7
```

```
        elif (float(country.new_cases) > 23000 and float(country.new_cases) <␣
↪25000):
            marker = 800
            country.size = marker
            color_sel = "indianred"
            country.color = color_sel
            a = 0.6


        elif (float(country.new_cases) > 20000 and float(country.new_cases) <␣
↪23000):
            marker = 800
            country.size = marker
            color_sel = "maroon"
            country.color = color_sel
            a = 0.5

        elif (float(country.new_cases) > 18000 and float(country.new_cases) <␣
↪20000):
            marker = 800
            country.size = marker
            color_sel = "firebrick"
            country.color = color_sel
            a = 0.4

        elif (float(country.new_cases) > 15000 and float(country.new_cases) <␣
↪18000):
           marker = 800
           country.size = marker
           color_sel = "rosybrown"
           country.color = color_sel
           a = 0.4

        elif (float(country.new_cases) > 13000 and float(country.new_cases) <␣
↪15000):
           marker = 500
           country.size = marker
           color_sel = "brown"
           country.color = color_sel
           a = 0.5

        elif (float(country.new_cases) > 11000 and float(country.new_cases) <␣
↪13000):
           marker = 300
```

```
            country.size = marker
            color_sel = "orangered"
            country.color = color_sel
            a = 0.5

        elif (float(country.new_cases) > 9000 and float(country.new_cases) <␣
↪11000):
            marker = 190
            country.size = marker
            color_sel = "coral"
            country.color = color_sel
            a = 0.6

        elif (float(country.new_cases) > 7000 and float(country.new_cases) <␣
↪9000):
            marker = 160
            color_sel = "orange"
            country.color = color_sel
            a = 0.7

        elif (float(country.new_cases) > 5000 and float(country.new_cases) <␣
↪7000):
            marker = 150
            country.size = marker
            color_sel = "gold"
            country.color = color_sel
            a = 0.8

        elif (float(country.new_cases) > 3000 and float(country.new_cases) <␣
↪5000):
            marker = 130
            country.size = marker
            color_sel = "yellow"
            country.color = color_sel
            a = 0.8

        elif (float(country.new_cases) > 1000 and float(country.new_cases) <␣
↪3000):
            marker = 120
            country.size = marker
            color_sel = "khaki"
            country.color = color_sel
            a = 0.9

        elif (float(country.new_cases) > 500 and float(country.new_cases) <␣
↪1000):
```

```
            marker = 100
            country.size = marker
            color_sel = "paleturquoise"
            country.color = color_sel
            a = 1

        elif (float(country.new_cases) > 300 and float(float(country.
↪new_cases)) < 500):
            marker = 70
            country.size = marker
            color_sel = "mediumseagreen"
            country.color = color_sel
            a = 1

        else:
            marker = 50
            country.size = marker
            color_sel = "forestgreen"
            country.color = color_sel
            a = 1

    return epicenters
```

[10]:
```
ep = draw_world(updated)
```

```
Italy
---------
 Cases: 88431.0
 HDI: 0.892
 Weekly Cases: 60187.429
 Total Vaccinations Per 100 People: 226.47
 Fatality rate: 0.01001937744268126
 Govt Rating: 53.7
['41.871940', '12.567380']

Niue
---------
 Cases: 112173.0
 HDI: 0.0
 Weekly Cases: 68535.0
 Total Vaccinations Per 100 People: 164.55
 Fatality rate: 0.014873255698088719
 Govt Rating: 0.0
[]

South Africa
---------
```

```
 Cases: 24204.0
 HDI: 0.709
 Weekly Cases: 17694.429
 Total Vaccinations Per 100 People: 192.55
 Fatality rate: 0.022831801877534613
 Govt Rating: 37.96
['-30.559482', '22.937506']

United States
---------
 Cases: 185094.0
 HDI: 0.463
 Weekly Cases: 142533.714
 Total Vaccinations Per 100 People: 187.57999999999998
 Fatality rate: 0.015950679013856058
 Govt Rating: 47.455
['39.500240', '-98.350891']

Western Sahara
---------
 Cases: 844207.0
 HDI: 0.737
 Weekly Cases: 670105.0
 Total Vaccinations Per 100 People: 146.81
 Fatality rate: 0.012171276305283986
 Govt Rating: 0.0
[]
```

5 Plotting

The plot is created using the plotly library, normally the plot would be shown outside the notebook when the code is run, but I'm using an inline version for the notebook. If my main file - covid_driver.py is run, the graphs wil open in your default browser.

Since Plotly is being used, the list of objects had to be converted to back to 2d lists, and then to data frames, the two methods below do that

[11]:
```
def obj_to_list(lol):
    '''
        Converts a list of objects to a 2d list with each countries attributes␣
 ↪as row

        Parameters
        ----------
        lol : List of Objects
```

```
        Returns
        -------
        2 dimensional list .

     '''
    epicenters = []
    #setting a background for plot
    # img = plt.imread("worldmap.jpg")
    # Creating a list of all countries as rows
    countries = []
    c = []
    for country in lol:
        name = country.name
        c.append(name)
        new_cases = float(country.new_cases)
        c.append(new_cases)
        weekly_cases = float(country.new_cases_weekly)
        c.append(new_cases)
        total_cases = country.total_cases
        c.append(total_cases)
        total_vax = country.total_vax
        c.append(total_vax)
        fully_vax = country.fully_vax
        c.append(fully_vax)
        booster = country.booster
        c.append(booster)
        fatality = country.fatality_r
        c.append(fatality)
        govt = country.govt
        c.append(govt)
        beds = country.beds
        c.append(beds)
        hdi = country.hdi
        c.append(hdi)
        lats = country.lats
        if lats != []:
            lat = float(lats[0])
            c.append(lat)
            long = float(lats[1])
            c.append(long)
        else:
            c.append(float(0))
            c.append(float(0))

        col = country.color

        if col != "":
```

```
            c.append(col)
        else:
            c.append("white")



        s = country.size
        if s != 0:
            c.append(s)
        else:
            c.append(0)

        countries.append(c)
        c = []

    return countries
```

[12]:
```
def obj_to_list2(lol, extra):
    '''
        Converts a list of objects to a 2d list with each countries attributes␣
↪as row

        Parameters
        ----------
        lol : List of Objects


        Returns
        -------
        2 dimensional list .

     '''
    epicenters = []
    #setting a background for plot
    # img = plt.imread("worldmap.jpg")
    # Creating a list of all countries as rows
    countries = []
    c = []
    count = 0
    for country in lol:

        name = country.name
        c.append(name)
        new_cases = country.new_cases
        c.append(new_cases)
        weekly_cases = float(country.new_cases_weekly)
        c.append(new_cases)
```

```
        total_cases = country.total_cases
        c.append(total_cases)
        total_vax = country.total_vax
        c.append(total_vax)
        fully_vax = country.fully_vax
        c.append(fully_vax)
        booster = country.booster
        c.append(booster)
        fatality = country.fatality_r
        c.append(fatality)
        govt = country.govt
        c.append(govt)
        beds = country.beds
        c.append(beds)
        hdi = country.hdi
        c.append(hdi)
        lats = country.lats
        if lats != []:
            lat = float(lats[0])
            c.append(lat)
            long = float(lats[1])
            c.append(long)
        else:
            c.append(float(0))
            c.append(float(0))

        col = country.color

        if col != "":
            c.append(col)
        else:
            c.append("white")



        s = country.size
        if s != 0:
            c.append(s)
        else:
            c.append(0)


        ex = extra[count]
        c.append(ex)

        countries.append(c)
        c = []
```

```
        count += 1

    return countries
```

This is the function which uses plotly to visually graph the covid-19 status of all countries.

[13]:
```
def plot_ep(world):
    '''

    plots epicenters on the world map
    Parameters
    ----------
    world : List
        List of country objects.

    Returns
    -------
    Returns a data frame with all country info .

    '''



    countries = obj_to_list(world)

    # Creating a data frame for countries
    my_data = pd.DataFrame(countries, columns = ['country', 'New_Cases',␣
↪'new_cases_weekly',
                                                 'Total_cases', 'total_vax',␣
↪'fully_vax', 'booster',
                                        'fatality_r', 'govt', 'beds', 'hdi',
                                        'lat', 'long', 'color', 'size'])

    # Doesn't matter which year only using it for geo location!
    df = px.data.gapminder().query("year == 2007")


    result = pd.merge(df, my_data, how = "inner", on = ["country"])


    print("Possible Current Epicenters are as Follows:" + "\n" +␣
↪"----------------" + "\n")
    epicenters = draw_world(world)
    ep = {}
    for country in epicenters:

        ep[country.name]= "Red Alert"
```

```
    scale = ["red", "crimson", "orange", "yellow", "khaki", "green"]
    scale.reverse()

    colorscales = px.colors.make_colorscale(scale)

    fig = px.scatter_geo(result, locations="iso_alpha",
                         size="size", color = "size", text = "New_Cases",
                         title="layout.hovermode='closest' (the default)",
                         color_continuous_scale = colorscales, hover_name =␣
↪"country",
                         projection="natural earth",
                         hover_data={'iso_alpha':False, # remove species from hover␣
↪data
                                     'New_Cases': True, # customize hover for column of␣
↪y attribute
                                     'fatality_r':True,
                                     "size" : False})


    fig.layout.coloraxis.colorbar.title = 'Epicenter Scale <br><i>Redder means␣
↪higher threat</i> <br></br>   '

    sizeref = 2 * max(result['size'])/(80**2)
    fig.update_traces(mode='markers', marker = dict(sizemode = 'area',
                                                    sizeref = sizeref, line_width=2))

    fig.update_layout(title='Covid Epicenters Today')

    iplot(fig)


    return result
```

[14]:
```
plot = plot_ep(updated)
# Returns a list of 'calculated' Epicenters
```

```
Possible Current Epicenters are as Follows:
----------------

Italy
---------
 Cases: 88431.0
 HDI: 0.892
 Weekly Cases: 60187.429
 Total Vaccinations Per 100 People: 226.47
 Fatality rate: 0.01001937744268126
```

```
 Govt Rating: 53.7
['41.871940', '12.567380']

Niue
---------
 Cases: 112173.0
 HDI: 0.0
 Weekly Cases: 68535.0
 Total Vaccinations Per 100 People: 164.55
 Fatality rate: 0.014873255698088719
 Govt Rating: 0.0
[]

South Africa
---------
 Cases: 24204.0
 HDI: 0.709
 Weekly Cases: 17694.429
 Total Vaccinations Per 100 People: 192.55
 Fatality rate: 0.022831801877534613
 Govt Rating: 37.96
['-30.559482', '22.937506']

United States
---------
 Cases: 185094.0
 HDI: 0.463
 Weekly Cases: 142533.714
 Total Vaccinations Per 100 People: 187.57999999999998
 Fatality rate: 0.015950679013856058
 Govt Rating: 47.455
['39.500240', '-98.350891']

Western Sahara
---------
 Cases: 844207.0
 HDI: 0.737
 Weekly Cases: 670105.0
 Total Vaccinations Per 100 People: 146.81
 Fatality rate: 0.012171276305283986
 Govt Rating: 0.0
[]
```

6 Computing Objective Two: Analyzing and Predicting Origin of Next New Variant

The data retrieved was in a tsv format, so I had to create a modified function for it.

[15]:
```
def read_tsv(filename):
    '''
    Function: read_csv
    Param: filename - which file should it read?
    Returns : a list of lists
    '''
    data =[]
    with open(filename, 'r') as infile:

        csvfile = csv.reader(infile, delimiter = "\t")
        next(csvfile)
        for row in csvfile:
            data.append(row)


        return data
```

So this is how it works:

Covid-19 Variant data is read - and the variant name is in this format: “hCoV-19///”

My goal was to understand which country has had the most instances of giving rise to variants of the disease. For that, I had to extract the “” second instance of the string, and then find which country that location falls under. To do that, I had to access a public API with all countries and their cities.

The following function: get_data() extracts data using API and cleans the dictionary so that we only have the country and city keys

The next function searches throught the sorted dict for the location and returns the country. compute(data, string) –> If the extracted string was Wuhan, compute(data, string) would return China.

[16]:
```
def get_data():
    '''

    Reading API data from web


    Returns
    -------
    Dictionary.

    '''
    url = "https://countriesnow.space/api/v0.1/countries"
```

```
    ids = requests.get(url)

    obj = ids.json()


    for k, v in obj.items():
        if k == "data":
            meta = v


    for d in meta:
        d.pop('iso2')
        d.pop('iso3')


    loc = ""
    data = {}
    for d in meta:
        for k, v in d.items():

            if k == "country":
                loc = v
                data[loc] ={}

            if k == "cities":

                data[loc] = v

    return data
```

[17]:
```
def compute(data, string):
    '''
    While checking variant origin location to database of countries,
    in order to assign country to that location .
    for ex, string will say Wuhan, function outputs China

    Parameters
    ----------
    data : dict
        Dictionary of read data from tsv.
    string : str


    Returns
    -------
```

```
    find : TYPE
        DESCRIPTION.

    '''
    # Dictionary : Key is country name, value is a list of their cities
    find = ""
    for k,v in data.items():
        c = clean_string(k)
        if string == c:
            find = k
        else:
            for city in v:
                cit = clean_string(city)
                if cit == string:
                    find = k
    return find
```

And then, read_variant(lol, data) counts the number of times a variant was discovered in a country. Giving us the frequency, i.e, historical data to justify or strengthen the probability that a new variant is possible.

[18]:
```
def read_variant(lol, data):
    '''
    Reads variant data, and sorts it as we need

    Parameters
    ----------
    lol : List of lists
        Each row is a variant type.

    Returns
    -------
    dictionary of country as key and number of declared variants as values


    '''

    var_data = {}

    countries = []
    for i in lol:

        strain = i[0]


        info = strain.split('/')
```

```
        check = info[1]

        string = clean_string(check)

        find = compute(data, string)

        countries.append(find)


        for country in countries:
            var_data[country] = countries.count(country)




    return var_data
```

[19]:
```
variant = read_tsv("Varaints.tsv")
data = get_data()
```

[20]:
```
var_most = read_variant(variant, data)
```

The function new_variant(lol, var_most) cross references this historical data with other factors research in comments) from the Covid-19 dataset to provide a prediction model of where the next new pandemic variant might orignate.

[21]:
```
def new_variant(lol, var_most):
    '''
    Calculates in which major epicenters's is there most likely a variant going␣
 ↪to develop

    Parameters
    ----------
    lol : List
        of objects - epicenters.

    Returns
    -------
    None.

    '''
    '''
    # A country is more likely to develop the variant is if:
        - Their chronic infection cases are high ( fatality rate is high )
        - Their ability to manage the population and quality of life : HDI
        - Their total cases are also very signifiantly large
```

```
        - their vaccination ratio is high
        Source: Reference #2
        '''
    prob = 0.0


# ================================================================================
#     HDI is divided into four tiers: very high human development (0.8-1.0),
#      high human development (0.7-0.79), medium human development (0.55-. 70),
#      and low human development (below 0.55).
#      Source: Reference # 3
# ================================================================================
    danger = []
    risk =[]

    for country in lol:

        if country.name in var_most:
            prob += 15
        if country.fatality_r > 0.015:
            prob += 20
        if float(country.hdi) < 0.55 or float(country.hdi) < 0.70:
            prob += 15
        if float(country.new_cases_weekly) > 18000:
            prob += 10
        if float(country.new_cases) > 10000:
            prob += 40
        if float(country.total_vax) > 185:
            prob += 10


        if prob > 70:
            danger.append(country)
            print(country, "\n",
                  "------------------",
                  "Probability of Variant Occurence:", prob, "\n",)
            risk.append(prob)

        prob = 0




    country = obj_to_list2(danger, risk)

    df = pd.DataFrame(country, columns = ['country', 'New_Cases',␣
↪'new_cases_weekly',
```

```
                                                'Total_cases',
                                        'total_vax', 'fully_vax', 'booster',
                                        'fatality_r', 'govt', 'beds', 'hdi',
                                        'lat', 'long', 'color', 'size', 'prob'])

    scale = ["red", "crimson","darkorange","orange", "yellow", "khaki"]
    scale.reverse()
    colorscales = px.colors.make_colorscale(scale)
    fig = px.bar(df, x='country', y='prob',
             hover_data=['New_Cases', 'fatality_r'], color='prob',␣
↪color_continuous_scale = colorscales,
             labels={'prob':'Probability of Variant Production in %'},␣
↪height=400)


    fig.layout.coloraxis.colorbar.title =  '<i>Redder means higher possibility</
↪i> <br> '


    iplot(fig)

    return danger
```

[22]:
```
new_variant(updated, var_most)
```

```
Ethiopia
---------
 Cases: 489458.0
 HDI: 0.485
 Weekly Cases: 308558.429
 Total Vaccinations Per 100 People: 193.19
 Fatality rate: 0.007863573292778788
 Govt Rating: 26.85
['9.145000', '40.489673']

 ------------------ Probability of Variant Occurence: 75

Haiti
---------
 Cases: 729335.0
 HDI: 0.51
 Weekly Cases: 556998.143
 Total Vaccinations Per 100 People: 198.77
 Fatality rate: 0.008118961803225352
 Govt Rating: 34.26
['18.971187', '-72.285215']
```

```
 ------------------ Probability of Variant Occurence: 75

United States
---------
 Cases: 185094.0
 HDI: 0.463
 Weekly Cases: 142533.714
 Total Vaccinations Per 100 People: 187.57999999999998
 Fatality rate: 0.015950679013856058
 Govt Rating: 47.455
['39.500240', '-98.350891']

 ------------------ Probability of Variant Occurence: 95
```

[22]:
```
[<country.Country at 0x7ffd787a2910>,
 <country.Country at 0x7ffd8b218d90>,
 <country.Country at 0x7ffd480fb7f0>]
```

7 To show that this model has worked accurately, recently a new genetic mutation or variation of the virus was detected in the US:

The variant was name hCoV-19/USA/NE-NPHL22-14797/2022, first detected on 2022-04-25, that is four days ago.

Data from Reference #5.

8 Computing Objective Three: Safety Ratings for each country

Moving to the final analysis from the poject

For the safety rating, I have used historical data of the country's international travel policies, and other factors from the larger covid-19 dataset like the ferocity of the infection - represented by the fatality rate, the human development index (which would signifiy access to resources like clean water, healthcare), number of hospital beds ber 1000 people (in case of an emergency, will the country's healthcare system provide for you?), and governemnt policy rating (this includes public transport, lockdowns, mask mandate measure, etc. It was a measure already calculated for in the dataset used).

[23]:
```
int_travel = read_csv("international-travel-covid.csv")
```

[24]:
```
def safety(lol, travel):
    '''
    Giving a safety rating to each country based on it's covid and healthcare␣
 ↪data
```

```
    Parameters
    ----------
    lol : list
        list of objects.
    graph: plotly graph object
        for adding sublots

    Returns
    -------
    plot.

    '''

    # Let's look at the countries covid restrictions to
    # insternational trvel in the last year
    # More open borders tend to lead to higher infections, variants and spread

    tdf = pd.DataFrame(travel, columns = ['country', 'code', 'date', 'rating'])



    convert_dict = {'country': str,
                'rating': float}

    check =  tdf.groupby('country')['rating'].describe()

    #Which was their most used policy
    double = check['top'].tolist()

    c = tdf['country'].unique().tolist()

    data = {}
    for i in range(0, len(c) - 1):

        data[c[i]] = double[i]



    ratings =[]
    rating = 0
    total = 0
    for country in lol:
        for k, v in data.items():
            if country.name == k and (v == 1) :
                rating += 20
            if country.name == k and (v == 2) :
                 rating += 10
```

```
        if country.fatality_r > 0.015:
            rating += 20
        if float(country.hdi) < 0.55:
            rating += 10
        if float(country.total_cases) > 1000000:
            rating += 10
        if float(country.total_vax) < 10:
            rating += 10
        if float(country.beds) < 3:
            rating += 5
        if float(country.govt) < 50:
            rating += 25
        total = 100 - rating
        ratings.append(total)
        rating = 0
        total = 0

    countries = obj_to_list2(lol, ratings)

    # Creating a data frame for countries
    my_data = pd.DataFrame(countries, columns = ['country', 'New_Cases',␣
↪'Total_cases',
                                                 'new_cases_weekly',
                                        'total_vax', 'fully_vax', 'booster',
                                        'fatality_r', 'govt', 'beds', 'hdi',
                                        'lat', 'long', 'color', 'size',␣
↪'rating'])



    scale = ["red", "crimson", "orange", "yellow", "khaki", "green"]


    colorscales = px.colors.make_colorscale(scale)


    result = pd.merge(tdf, my_data, how = "inner", on = ["country"])

    fig = px.bar(result, x= "country", y = "size", color = "rating_y",
                 color_continuous_scale= colorscales, hover_name= "rating_y")

    fig.update_layout( hovermode = "x unified")
    iplot(fig)

    countries = result["country"].tolist()
    rate = result["rating_y"].tolist()
```

```
    safe = {}
    print("To Summarize, countries which are safe to travel to are: \n␣
↪-------------")
    for i in range(0, len(countries) - 1):

        if rate[i] > 90:
         safe[countries[i]] = rate[i]

    for k, v in safe.items():

         print(k)
         print("With a safety rating of:", v, "\n")


    return safe
```

[25]:
```
safe = safety(updated, int_travel)
```

```
To Summarize, countries which are safe to travel to are:
 -------------
Azerbaijan
With a safety rating of: 100

Belize
With a safety rating of: 95

Bhutan
With a safety rating of: 95

Brunei
With a safety rating of: 95

Cape Verde
With a safety rating of: 95

China
With a safety rating of: 100

Fiji
With a safety rating of: 95

Kiribati
With a safety rating of: 95

Laos
With a safety rating of: 95
```

```
Libya
With a safety rating of: 100

New Zealand
With a safety rating of: 95

Oman
With a safety rating of: 95

Saudi Arabia
With a safety rating of: 95

Seychelles
With a safety rating of: 100

Solomon Islands
With a safety rating of: 95

Tonga
With a safety rating of: 95

Turkmenistan
With a safety rating of: 100

Vanuatu
With a safety rating of: 95
```

Finally to visualise your travel around the world after two years in quarantine, here is a simple graph which shows you top countries to avoid, and countries safe to travel to (if travel to them is open).

[26]:
```
def draw_safe(safe, ep):


    fig = make_subplots(start_cell="top-left")


    epicenters = obj_to_list(ep)
    # Creating a data frame for countries
    safe = list(safe.items())

    df_1 = pd.DataFrame(epicenters, columns = ['country', 'New_Cases',␣
↪'new_cases_weekly',
                                               'Total_cases', 'total_vax',␣
↪'fully_vax', 'booster',
                                          'fatality_r', 'govt', 'beds', 'hdi',
                                          'lat', 'long', 'color', 'size'])
```

```
df_2 = pd.DataFrame(safe, columns=["country", "rating"])


df = px.data.gapminder().query("year == 2007")


first = pd.merge(df, df_1, how = "inner", on = ["country"])
second = pd.merge(df, df_2, how = "inner", on = ["country"])


sizeref1 = 2 * max(second['rating'])/(20**2)
sizeref2 = 2 * max(first['size'])/(20**2)

fig.add_trace(
    go.Scattergeo(
  locations = second["iso_alpha"],
  marker_size = second["rating"],
  text = second["country"],
  mode="markers+text",
  marker_color = "green",
  marker = dict(sizemode = 'area',
  sizeref = sizeref1, line_width=0, line_color = "green",
  symbol = 'arrow-bar-down'),
  textposition = "middle center",
  name = 'Safe to Travel'))

fig.add_trace(
    go.Scattergeo(
  locations = first["iso_alpha"],
  marker_size = first["size"],
  text = first["country"],
  mode="markers+text",
  marker_color = "red",
  marker = dict(sizemode = 'area',
  sizeref = sizeref2, line_width=2,
  symbol = 'x'),
  textposition="bottom center",
  name = "Epicenters"),
  )



fig.update_layout(hovermode = "x unified")
fig.update_layout(title='Travelling during a slowing pandemic')
```

```
    iplot(fig)
```

[27]:
```
draw_safe(safe, ep)
```

References 1. All Code Related to Plotly: https://plotly.com/python/ 2. What Makes Covid Variants Emerge: https://ec.europa.eu/research-and-innovation/en/horizon-magazine/covid-19-variants-five-things-know-about-how-coronavirus-evolving 3. Human Development Index Measures : https://worldpopulationreview.com/country-rankings/hdi-by-country 4. GitHub Link to main covid dataset and it's auxillary set like the travel policy data: https://github.com/owid/covid-19-data/tree/master/public/data 5. For understanding variant data and analysis: https://www.gisaid.org/epiflu-applications/influenza-genomic-epidemiology/ 6. API database and code: https://documenter.getpostman.com/view/1134062/T1LJjU52#dd5bd0d9-2602-4161-8c77-3af30cd2f41a 7. Pandas documentation: https://pandas.pydata.org/docs/reference/api/pandas.DataFrame.sort_v

9 Thank you!! Hope you enjoyed the project!

[ ]:
```

```